



1. 常用汇编语句

1.1 数据传送指令

mov指令

格式: mov 操作数1, 操作数2

```
mov eax, ebx  
mov byte ptr [var], 5
```

将第二个操作数复制到第一个操作数

mov指令的两个操作数不可同时为内存





1. 常用汇编语句

1.1 数据传送指令

push指令

```
push eax  
push [var]
```

将操作数压入栈中，常用于函数调用和现场保护
ESP是栈顶，入栈前先将ESP减4（栈向低地址方向增长）
再将操作数压入ESP所指地址





1. 常用汇编语句

1.1 数据传送指令

pop指令

从栈中弹出数据

pop eax

pop [var]

从栈中弹出数据

出栈前先将ESP所指地址的内容读出，再将ESP加4





1. 常用汇编语句

1.2 算术和逻辑运算指令

add/sub指令

```
sub eax,10  
add byte ptr [var],10
```

add指令将两个操作数相加
sub指令将第一个操作数减去第二个操作数
结果均保存在第一个操作数中





1. 常用汇编语句

1.2 算术和逻辑运算指令

inc/dec指令

```
dec eax
```

```
inc dword ptr [var]
```

inc指令对操作数执行自增1操作

dec指令对操作数执行自减1操作





1. 常用汇编语句

1.2 算术和逻辑运算指令

imul指令

```
imul eax,[var]  
imul esi,edi,25
```

imul指令是带符号乘法指令
支持两种格式：

- (1) 双操作数：将第一、第二操作数相乘，结果存入第一个操作数（必须为寄存器）
- (2) 三操作数：将第二、第三操作数相乘，结果存入第一个操作数（必须为寄存器）





1. 常用汇编语句

1.2 算术和逻辑运算指令

idiv指令

```
idiv ebx  
idiv dword ptr [var]
```

idiv指令是带符号除法指令，仅指定除数
被除数为edx:eax组成的64位有符号数
执行后，商存入eax，余数存入edx





1. 常用汇编语句

1.2 算术和逻辑运算指令

and/or/xor指令

寄存器寻址

```
and eax,0fH
```

```
xor edx,edx
```

and是按位与操作

or是按位或操作

xor是按位异或操作

结果存入第一个操作数





1. 常用汇编语句

1.2 算术和逻辑运算指令

not指令

国图公共空间
永久保留1981087000

```
not btye ptr [var]
```

按位取反指令





1. 常用汇编语句

1.2 算术和逻辑运算指令

neg指令

取反指令
永久保留1981087000

neg eax

取负指令





1. 常用汇编语句

1.2 算术和逻辑运算指令

shl/shr指令

移位指令

```
shl eax,1  
shr ebx,cl
```

shl指令是逻辑左移指令

shr指令是逻辑右移指令

第一个操作数为被移位数，第二个操作数为移位位数





1. 常用汇编语句

1.3 控制流指令

jmp指令

跳转到指定地址继续执行

```
jmp begin
```

无条件转移到label标签所指示的地址继续执行





1. 常用汇编语句

1.3 控制流指令

jcondition指令

je <label>	相等时转移
jz <label>	结果为0时转移
jne <label>	不相等时转移
jg <label>	大于时转移
jge <label>	大于等于时转移
jl <label>	小于时转移
jle <label>	小于等于时转移

根据程序状态字寄存器（PSW）中的状态标志决定是否转移





1. 常用汇编语句

1.3 控制流指令

cmp/test指令

```
cmp dword ptr [var],10  
jne loop
```

```
test eax,eax  
jz xxxx
```

cmp指令执行减法运算但不保存结果，仅根据结果设置标志位
test指令执行按位与运算但不保存结果，仅更新标志位
这两个指令通常和jcondition指令配合使用





2. 选择语句的机器代码表示

```
int get_cont(int *p1,int *p2){  
    if (p1>p2)  
        return *p2;  
    else  
        return *p1;  
}
```

```
mov eax, dword ptr [ebp+8]    #R[eax]←-M[R[ebp]+8], 即加载参数p1  eax  
mov edx, dword ptr [ebp+12]  #R[edx]←-M[R[ebp]+12], 即加载参数p2到edx  
cmp eax, edx                  #比较p1和p2, 根据结果设置条件码  
jbe .L1                       #若p1≤p2, 则转移到L1  
mov eax, dword ptr [edx]      #R [eax]←M[R [edx]], 即取*p2作为返回值  
jmp .L2                       #无条件转移到L2, 跳过 else 分支  
.L1:  
mov eax, dword ptr [eax]      #R [eax]←M[R [eax]], 即取*p1 作为返回值  
.L2:
```





3. 循环语句的机器代码表示

```
int nsum_for (int n){  
    int i;  
    int result = 0;  
    for (i=1; i<=n;i++)  
        result +=i;  
    return result;  
}
```

```
mov ecx, dword ptr [ebp+8] #R[ecx]-M[R[ebp]+8], 即加载参数n到 ecx  
mov eax, 0                 #R[eax]-0, 即初始化result=0  
mov edx, 1                 #R[edx]-1, 即初始化i=1  
cmp edx, ecx               #比较i与n(R[edx]与R[ecx])并设置标志位  
jg .L2                    #If greater, 则转移到 L2  
.L1:                      #循环体入口  
add eax, edx               #R[eax]-R[eax]+R[edx],即result += i  
add edx, 1                 #R[edx]<-R[edx]+1, 即i++  
cmp edx, ecx               #再次比较i:n  
jle .L1                   #If less or equal, 则转移到 L1  
.L2:                      #循环结束点
```





4. 过程调用的机器代码表示

过程P（调用者）

过程Q（被调用者）

- 后续关注公众号「研大」
- (1) **P**将入口参数（实参）放到**Q**能访问的位置
 - (2) **P**执行call指令，将返回地址（call的下一条指令地址）压入栈中，并转移到**Q**
 - (3) **Q**建立自己的栈帧，为局部变量分配空间，并在必要时保存某些寄存器
 - (4) **Q**执行其主体代码
 - (5) **Q**将返回结果放到约定位置，释放局部变量空间，并恢复之前保存的寄存器
 - (6) **Q**执行ret指令，从栈顶弹出返回地址，并转移回**P**继续执行





4. 过程调用的机器代码表示

过程P（调用者）

过程Q（被调用者）

```
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1,temp2);  
    return sum;  
}
```

调用Q之前，P自己可能已经在用一些寄存器，比如EAX、EBX、ECX等
但是Q运行时也要用寄存器，如果Q直接把这些寄存器改了，等Q返回P时，P原来寄存器里的
值就丢了，程序就可能出错
所以需要规定：哪些寄存器由调用者保护，哪些寄存器由被调用者保护





4. 过程调用的机器代码表示

过程P（调用者）

```
push eax
push ecx
push edx

call Q

pop edx
pop ecx
pop eax
```

后续关注公众号[研大]
永久微信1951657888

EAX、ECX、EDX是调用者保存寄存器

如果P调用Q，并且P调用完Q后还想继续使用EAX、ECX、EDX里的旧值，那么P自己要在调用Q之前保存它们

一般将这些内容保存在栈里





4. 过程调用的机器代码表示

过程Q（被调用者）

Q:

```
push ebx
push esi
push edi
```

Q自己的代码

```
pop edi
pop esi
pop ebx
ret
```

后续关注公众号「研大」
永久微信1871457888

EBX、ESI、EDI是被调用者保存寄存器

如果Q想使用EBX、ESI、EDI

那么Q自己必须先保存，返回前恢复
调用者P默认相信它们在调用前后不会变
如果Q要改，就必须自己恢复

一般将这些内容保存在栈里





4. 过程调用的机器代码表示

过程P（调用者）

过程Q（被调用者）

后续关注公众号[研大]
微信号1951657888

```
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1,temp2);  
    return sum;  
}
```

每次过程（函数）被调用时，系统都会在栈中为其分配一块独立的存储区域，用于保存局部变量、返回地址、参数等信息，这块区域称为**栈帧**





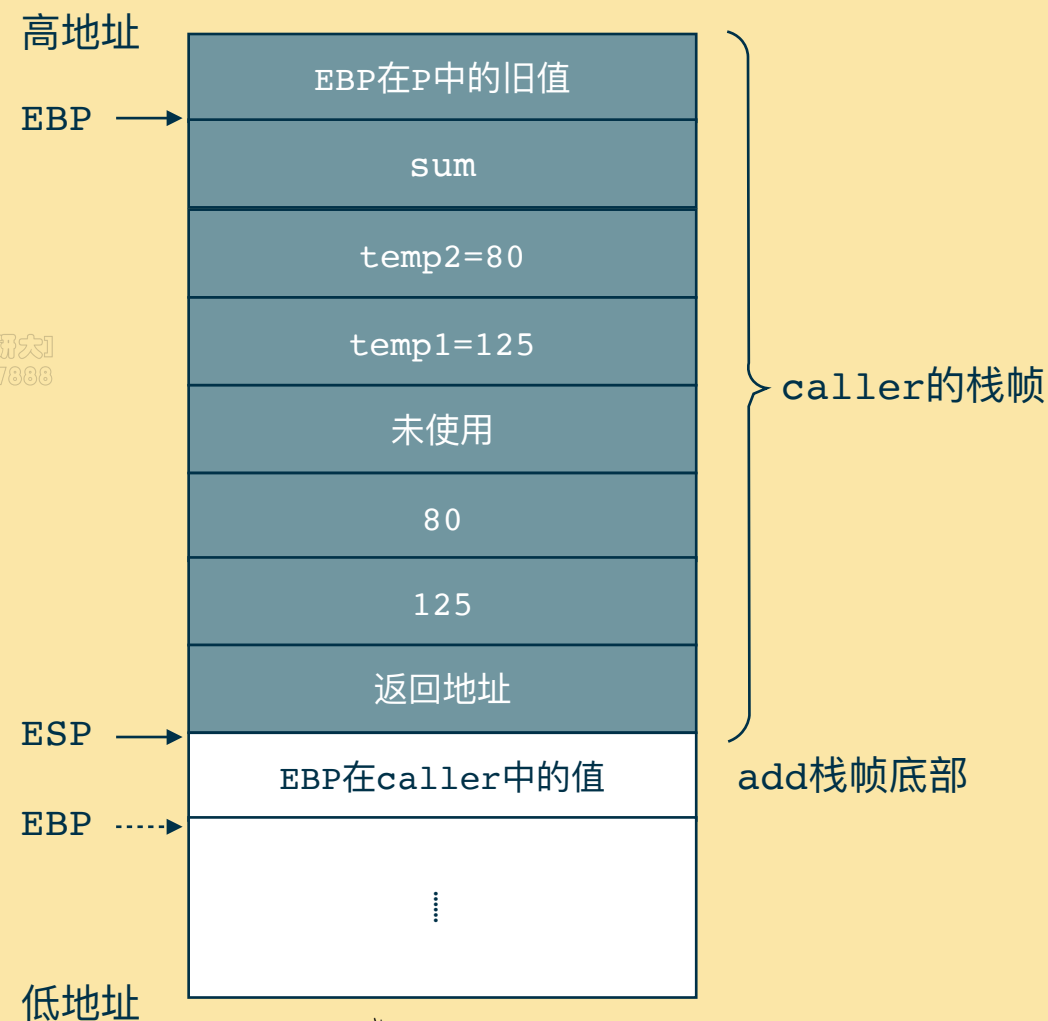
4. 过程调用的机器代码表示

```
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1,temp2);  
    return sum;  
}
```

ESP (Stack Pointer) 栈顶指针
永远指向当前栈顶

EBP (Base Pointer) 栈帧指针
指向当前函数栈帧的基准位置

后续关注公众号「研大」
永久微信1951657888



4. 过程调用的机器代码表示



```
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1,temp2);  
    return sum;  
}
```

假设caller被过程P调用
caller又调用了add

caller:

push ebp

保存调用者P的EBP

mov ebp,esp

建立新栈帧, EBP←当前栈顶

sub esp,24

为局部变量和参数区分配24B

mov [ebp-12],125

temp1=125

mov [ebp-8],80

temp2=80

mov eax,[ebp-8]

eax←temp2

mov [esp+4],eax

temp2放入参数区高地址

mov eax,[ebp-12]

eax←temp1

mov [esp],eax

temp1放入参数区低地址

call add

调用add, 返回值保存在eax

mov [ebp-4],eax

sum=eax

mov eax,[ebp-4]

返回sum

leave

恢复栈帧(=mov esp,ebp; pop ebp)

ret

返回调用者



4. 过程调用的机器代码表示



```
int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1,temp2);
    return sum;
```

假设caller被过程P调用
caller又调用了add

caller:

```
push ebp
mov  ebp,esp
sub  esp,24
```

保存调用者P的EBP
建立新栈帧, EBP←当前栈顶
为局部变量和参数区分配24B

```
mov  [ebp-12],125
mov  [ebp-8],80
```

temp1=125
temp2=80

```
mov  eax,[ebp-8]
mov  [esp+4],eax
```

eax←temp2
temp2放入参数区高地址

```
mov  eax,[ebp-12]
mov  [esp],eax
```

eax←temp1
temp1放入参数区低地址

```
call add
```

调用add, 返回值保存在eax

```
mov  [ebp-4],eax
mov  eax,[ebp-4]
```

sum=eax
返回sum

```
leave
ret
```

恢复栈帧(=mov esp,ebp; pop ebp)
返回调用者

